Design and Architecture

Target company search over ~50,000 companies. Natural-language queries in, ranked companies out, sub-200ms, deployable to a free-tier container.

This document explains why the system is built the way it is. Where a decision had a real alternative, the alternative is named and the trade is stated. Measurements throughout are from a 6-core i5-9400F. The full measured run is in `reports/self_assessment.md`, regenerated with `python -m scripts.benchmark`.

---

1. The corpus decides the architecture

Before writing any code I profiled `companies.json`. The result reshaped everything that followed, so it comes first.

```
50,000 records · 8 fields · zero nulls · ids 1..50000 contiguous
10 industries · 8 countries · 6 revenue buckets · founded 1995-2024
employee_count 5..5000
```

The important part is the text. **49,999 of the 50,000 descriptions are generated from one template:**

```
"{modifier} {noun} for {topic}."

modifier ∈ {AI-powered, data-driven, machine learning, real-time,
            cloud-native, automated, distributed, predictive}     (8)
noun     ∈ {platform, solution, engine, software, system,
            infrastructure}                                        (6)
topic    ∈ 27 distinct values
```

Total description vocabulary: **80 words**. The remaining record is one of twelve hand-written descriptions at the head of the file.

Three consequences drove the design:

The head carries no signal. "AI-powered platform" appears ~1,000 times, distributed evenly across all ten industries. A bag-of-words retriever scoring the raw description spends most of its evidence on noise. One of the brief's own example queries — *"Education companies in the UK using AI for personalized learning"* — contains "AI", which matches ~5,000 companies in every sector. Another contains "infrastructure", matching ~8,000. Both had to be actively removed from the lexical query, or they would swamp the real signal.

Topic is the only discriminator, and it is a closed set of 27. That means it does not have to be inferred fuzzily at query time. It can be extracted once, at ingest, and stored as a structured facet — converting the dominant part of relevance from approximate scoring into exact set intersection. Faster and more accurate at the same time.

Every topic implies exactly one industry. All 27 are 100% industry-pure (verified across all 50,000 rows). So "working on drug discovery" implies Biotech without the user saying so, and — more usefully — a topic can disambiguate an ambiguous industry mention. *"Automotive software companies … autonomous driving"* matches both Automotive and Technology on the word "software"; the topic settles it.

Two of the brief's own queries are unsatisfiable

Employee counts are heavily skewed: **90.4% of companies have 500+ employees**, and only 147 have fewer than 20. Cross-tabulating the example queries against the data:

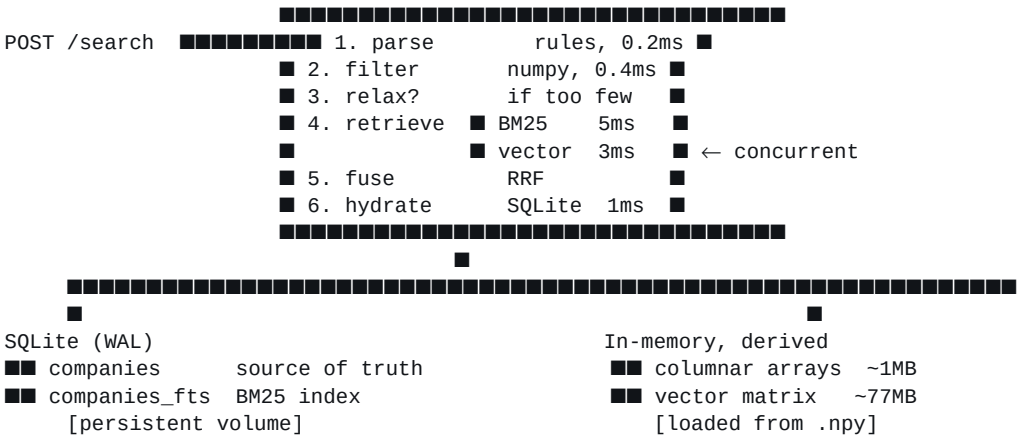
Query	Strict matches
Q7 — UK telecom, 5G analytics, **<20 employees**	**0**
Q14 — German, founded >2018, **<100 employees**, drug discovery	**1**

Q7 has no answer at all. Q14 has exactly one (Helix BioCompute — 95 employees, founded 2019). A system that returns an empty page for Q7 is *correct* and *useless*. Section 6 covers what it does instead.

> *Q14 was initially measured as zero. That was wrong, and the error is instructive: my first ground-truth script extracted topics with a regex on the template, so for the hand-written "AI-powered drug discovery and molecular analysis platform for biotech research teams" it recorded the topic as "biotech research teams" and lost "drug discovery". The shipped extractor scans for **all** topic mentions, which is why that company is now found.*

---

2. Architecture



The split is deliberate: **one durable store, everything else derived and rebuildable.** SQLite holds the truth. The columnar arrays and the vector matrix are caches that can be regenerated from it at any time. That keeps the stateful surface small — which is what makes restarts, redeploys and volume mounts boring.

Request path, and why filtering is first

Filtering costs 0.4ms and produces an exact match count. Running it before retrieval means:

- the vector search scans a smaller candidate set and gets **faster** as filters tighten (0.06ms at high selectivity versus 2.9ms unfiltered);
- `total` in the response is exact, not an estimate;
- the two retrievers can run concurrently against the same mask, so a request costs `max(embed+vector, bm25)` rather than their sum.

---

3. Storage

SQLite, not Postgres

The latency budget is 200ms and the query embedding alone spends ~25ms of it. A managed Postgres on a free tier adds a network round trip on every query and, on the plans that are actually free, suspends when idle — a multi-second cold start on the first query after a quiet period. An embedded database has neither problem: reads are served from the page cache in the same process.

What is given up, stated plainly: no horizontal write scaling, one node, and the durable state is a file on a volume rather than a managed service with automated backups. At 50k rows on a single container, none of those bind. Section 8 covers the point where they do.

Option	Why not
Managed Postgres + pgvector	Network hop plus free-tier cold starts, both larger than the entire query budget
Elasticsearch / OpenSearch	Wants more RAM for one node than the whole service is budgeted; no usable free tier
Dedicated vector DB (Qdrant, Pinecone)	A second network hop for a 2.5ms operation
In-memory only	No durability; a restart would mean a 6-minute rebuild

Configuration that matters: `journal_mode=WAL` so the ingestion endpoint can write while searches read, `synchronous=NORMAL` (a crash can lose the last transaction but never corrupts, and the corpus is re-ingestable), and a per-connection 64MB page cache.

Concurrency

A pool of read connections plus one serialised writer, mirroring what WAL actually permits. All through `aiosqlite`, which runs each connection on its own thread.

This is not incidental. A synchronous `sqlite3` call inside a request handler blocks the event loop for its whole duration — a 3ms FTS scan under 50 concurrent requests becomes 150ms of head-of-line blocking for everyone. Moving the work onto connection threads keeps the loop responsive and lets reads genuinely overlap.

Keyword index

SQLite FTS5, external-content, over `name`, `description` and `topics_text`, with BM25 column weights of 2 / 1 / 4. Topics weigh most because they are the distilled signal; name above description because a company called "GridPulse Analytics" matching "grid" says more than the same word in boilerplate.

Two details that are easy to get wrong:

- **FTS5's MATCH grammar treats `-`, `*`, `"`, `:`, `^`, `AND`, `OR` and `NEAR` as operators.** User text goes straight into that grammar. Unescaped, "e-commerce" parses as a NOT and returns silently wrong results. Every term is stripped of syntax and quoted; ten hostile inputs are covered by tests.
- `bm25()` **returns negative scores**, more negative being better. The sign is flipped once at the boundary so every retriever in the system agrees that higher is better.

Columnar filter arrays

Filterable fields are held as parallel numpy arrays in the same row order as the vector matrix, because the vector retriever needs a boolean mask over row positions — not a set of ids from SQL. Round-tripping ids out of SQLite and back into numpy per request would cost more than the search.

Details worth stating:

- Category predicates use a boolean lookup table indexed by the code column rather than `np.isin`, which sorts its argument and binary-searches per row. Measured: a six-constraint filter went from **1.10ms to 0.40ms**.
- Topics use postings lists, not a dense matrix. Membership is many-to-many; a rows × topics boolean matrix works at 27 topics but a 500-topic taxonomy over 5M companies would be 2.5GB.
- Missing numerics are `-1` and every predicate also requires `>= 0`. A company with an unknown founding year must not satisfy "founded before 2005" — **absence of data is not evidence of a small number.**

---

4. Search

Query understanding: rules, not an LLM

An LLM parser would handle more phrasings. It would also add 300-800ms to a 200ms budget, put a network dependency and a per-query cost on the hot path, and return different output for the same input on different days.

The queries this system receives draw on a closed vocabulary: 8 countries, 10 industries, 27 topics, and numeric predicates over three fields. Rules cover that in **0.15ms p50**, deterministically, and every decision is inspectable.

The pipeline consumes tokens as it goes — numbers first (so "500M" cannot be re-read as something else), then topics, industries, locations, with the residue becoming the lexical query. Consumed positions act as barriers, which fixed a real bug: in *"founded after 2018 with fewer than 100 employees"*, the backward scan for "fewer than" reached past its own clause, found the `founded` belonging to the previous constraint, and filed the headcount limit under founding year.

The honest limit: phrasings outside the alias tables do not become filters. They fall through to lexical and semantic retrieval, so the query still returns sensible results — it just loses the hard constraint. If usage showed a long tail of unparsed queries, the fix is an LLM parser behind a cache keyed on the normalised query, with these rules as the synchronous fallback. The `ParsedQuery` contract would not change.

The parse is returned in every response. A user who types a sentence and receives ten companies otherwise cannot tell whether "founded after 2015" was understood or silently dropped. It also gives the UI editable filter chips, and explicit filters in the request override the parse so a correction sticks.

Semantic retrieval: exact, in-process, no ANN

Embeddings are `BAAI/bge-small-en-v1.5` (384-d) on onnxruntime via fastembed — not sentence-transformers on PyTorch, which would add ~2GB to the image and seconds to every cold start for no quality gain at this model size.

Exact brute force rather than an approximate index. The measurement that settles it:

Query embedding	**25 ms**
Exact top-k over 50,000 × 384	**2.9 ms**

HNSW would cut the 2.9ms to perhaps 0.3ms and leave the 25ms untouched: a ~1% end-to-end gain for a C++ dependency, a build step, tuning parameters and approximate recall. It also composes badly with filtering, which most queries here carry — pre-filtering an HNSW graph either disconnects the reachable neighbourhood or forces over-fetching, whereas a masked linear scan is exact by construction and gets *faster* as filters tighten.

Within the scan there is a second choice, made by measurement rather than intuition. Below a selectivity threshold it is cheaper to gather the surviving submatrix; above it, one full matmul beats copying most of the matrix:

Selectivity	Rows	Full	Gather
0.1%	50	3.06 ms	**0.05 ms**
5%	2,500	3.10 ms	**1.50 ms**
12%	6,000	3.46 ms	3.36 ms ← crossover
20%	10,000	**3.09 ms**	5.62 ms
80%	40,000	**2.96 ms**	20.64 ms

Threshold set at 10%, clear of the noisy boundary. My initial guess had been 20%, which would have been consistently wrong for the 12–20% band.

What gets embedded is composed, not raw. Given the boilerplate head, embedding the description verbatim puts most of each vector into noise. The embedded text is:

```
"{name}. {description} Focus: {topics}. Sector: {industry}."
```

Location is deliberately **excluded**. It is already an exact filter, and including it would make a Finnish fintech more similar to a Finnish biotech than to a German fintech — wrong for topical similarity, and it would corrupt `/similar`, whose entire job is "who else does what this company does".

Does the semantic layer earn its place on a corpus this templated? Tested with phrasings that have no alias in the taxonomy:

Query	Top topic
"companies preventing financial crime and money laundering"	fraud detection
"software that helps hospitals keep track of patients remotely"	patient monitoring
"firms helping utilities predict how much power people will use"	smart grid
"tools for finding new medicines"	drug discovery

72/80 correct topics in the top 10 across eight such queries. Yes — it earns it.

Fusion

Reciprocal Rank Fusion over the two ranked retrievers:

```
score(d) = Σ  w / (k + rank(d)),   k = 60
```

Not a weighted sum of raw scores: BM25 is unbounded and corpus-relative, cosine is bounded [-1, 1]. Any fixed blend of the two is arbitrary and drifts as the corpus changes. Ranks are directly comparable.

Topic overlap is added as a third **bounded** term rather than a third ranked list — as a list it would be almost all ties, since thousands of companies share any single topic. Scaled by `1/k`, it is worth roughly a first-place finish in one retriever: influential, not decisive.

RRF's strength — ignoring magnitude — is also its blind spot, and it produced a concrete bug. Searching a company by name returned a *different* company, because the exact name match sat at rank 0 of the lexical list and a mediocre semantic match sat at rank 0 of the vector list; identical RRF scores, and the tie fell through to company id. Raw scores now break ties (keyword first, since an unbounded BM25 hit on a rare term is higher-precision evidence than a cosine value that is never far from its neighbours), without being added to the fused score. The RRF ordering itself is untouched.

Boolean semantics for topics

Multiple topics are **OR-ed with a rank boost**, not AND-ed. This contradicts a literal reading of *"focused on diagnostics **and** patient monitoring"*.

It is still right. All but eleven companies in this corpus carry exactly one topic, so strict conjunction returns zero results for that query and three others in the brief. The user means "in the diagnostics / patient-monitoring space", and OR-with-boost delivers that while still ranking genuine both-topic matches first.

---

5. Similarity

`GET /companies/{id}/similar` reuses the seed's stored document vector, so it makes no embedding call at all — **8ms against 46ms** for a cold text query, the fastest path in the API.

Because location is excluded from the embedding, similarity is topical: the nearest neighbours of a Finnish fraud-detection company are fraud-detection companies in Sweden, the UK and France. `same_industry` and

`same_location` constrain it explicitly when that is what is wanted.

With no vector index loaded, it falls back to ranking by shared topics — still a reasonable notion of similarity in this corpus, and a working endpoint rather than a 503.

---

6. Handling unsatisfiable queries

Q7 has no answer. Returning `[]` is correct and looks broken.

Constraints are dropped one at a time, **least central first**:

`employees → revenue → founded → industry`

Location and topic are never dropped. They are the substance of the request; returning German biotech to someone asking for Finnish fintech under a "relaxed" banner is worse than an honest empty page. Numeric bounds are usually approximate in the user's head ("fewer than 20 employees" means "small"), and industry is frequently redundant because the topic implies it.

Three properties make this trustworthy rather than merely convenient:

1. **The response always says what was surrendered** — `relaxation.dropped`, `strict_result_count`, and a human-readable message. Silently widening a search is how a system loses a user's trust.
2. **Exact matches lead.** Q14's single qualifying company was, before this, buried among 53 approximate ones — worse than not relaxing at all. It now ranks first with the reason `"matches every requested constraint"`.
3. **Near-misses outrank far-misses.** "Fewer than 20 employees" surfaces the 37- and 114-person companies ahead of the 3,259-person one. Employees scale relatively; years scale absolutely — normalising a year distance by 2018 makes every distance ≈0 and hands everyone the full bonus, which was a bug caught in testing.

`allow_relaxation: false` disables it for callers wanting exact-only semantics.

---

7. Latency

Budget: 200ms. Measured end to end through the ASGI stack (`reports/self_assessment.md`, regenerate with `python -m scripts.benchmark`):

Stage	ms (warm)	Note
Parse	0.21	rules
Filter	0.38	numpy over 50k
Retrieve	5.71	BM25 and vector, concurrent
Fuse + rank	0.16	
Hydrate	1.02	SQLite, one round trip
Total, warm	**10.5 p50 · 14.2 p95**	embedding cached
Total, cold	**46.3 p50 · 49.8 p95**	unseen query, full encode
`/similar`	**8.1 p50**	no embedding call at all

Both totals are wall-clock at the client, so they include request parsing, validation and JSON serialisation — not just the search. The server's own `took_ms` for a cold query is ~37ms; the remainder is HTTP framing.

Even the cold p99 (50.7ms) sits at a quarter of the budget, and the p99 under 16 concurrent requests (192ms) stays inside it.

The dominant cost is the embedding, at roughly ten times the vector search it feeds. Two things follow.

Cache query embeddings. They are a pure function of the query string. An LRU turns a repeat query into a ~2µs lookup, taking a request from 46ms to 10ms. Search traffic is heavily repetitive, so this is the highest-leverage cache in the system.

Never block the event loop on it. 25ms of inline CPU stalls every other request on the worker; at 50 concurrent the tail becomes seconds. Embedding runs on a thread, and onnxruntime releases the GIL, so this is real parallelism.

`embed_threads=1` was measured, not assumed:

Threads	Concurrency	p50	p95	RPS
1	1	26.6 ms	29.2 ms	36.9
all (6)	1	33.5 ms	52.7 ms	27.6
1	8	100.3 ms	**167.7 ms**	**73.0**
all (6)	8	109.4 ms	300.7 ms	52.1

One thread per inference wins at every level. Fanning a small inference across cores mostly buys coordination overhead, and under load the requests contend for the same cores.

Startup

The brief calls out frequent restarts, so nothing expensive happens at boot. Embedding 50k documents takes ~6 minutes and is done **at image build time**; the container ships a finished index. Startup is: open SQLite, build columnar arrays (~0.5s), load a 77MB `.npy`, load the model. **~1.5s total.**

The cost is a ~900MB image. Pulling that once per deploy beats six minutes of unavailability per container start, and it removes a runtime dependency on Hugging Face being reachable.

---

8. Scaling

The current design is right for 50k on one container. Here is where each part breaks and what replaces it.

Scale	What breaks first	Change
50k (today)	—	As built. 77MB vectors, ~1.5s start.
500k	Nothing structural. Vectors ~770MB — too large to hold comfortably.	Quantise to int8 (~190MB, ~1% recall loss) or move to `float16`. Exact scan is ~30ms; still inside budget.
1–5M	Exact scan exceeds the budget (~300ms at 5M).	Introduce ANN behind the existing `VectorIndex.search(query, k, mask)` interface — hnswlib in-process, or Qdrant if the index no longer fits in RAM. Pre-filtering needs care; over-fetch and post-filter.
10M+	Single-node SQLite writes; whole-corpus rebuild on ingest.	Postgres or a managed store as source of truth; vectors in a dedicated service; shard by region or industry. The API contract does not change.

Scale	What breaks first	Change
360M (production)	Everything single-node.	Shard the corpus; a query fans out to shard replicas and merges by RRF, which is rank-based and therefore merges across shards correctly without score normalisation.

The interfaces were kept narrow specifically so these swaps are local: `VectorIndex.search()`, `KeywordIndex.search()`, `ColumnStore.mask()`. Each is one file.

The taxonomy is the part that does not generalise as-is. 27 hand-written topics work because the corpus has 27. A real corpus has open-ended descriptions, so the vocabulary would be *derived*: cluster the descriptions, label the clusters with an LLM offline, persist the result in the same structure. The search path is unchanged — it still consumes a topic vocabulary with aliases. Only the way the vocabulary is produced changes, which is why it is isolated in one module.

Ingestion at scale. Today a write rebuilds the whole column store (~0.5s at 50k), which is fine for occasional writes and much simpler than incremental array maintenance. At millions of rows it needs to become incremental, or batched behind a queue with periodic rebuilds. New companies are searchable by keyword and filters immediately; they enter the vector index at the next build. The API states this in `semantic_indexed` rather than leaving it to be discovered.

---

9. Reliability

Degradation, not failure. Every dependency below the durable store is optional at runtime:

Missing	Behaviour
Vector matrix	Lexical + filters. `semantic_search: false` in `/health`.
Embedding model	Same.
Stale/foreign vector index	Detected by model name and row coverage, discarded, logged.
Corpus	Process starts, `/health/ready` returns 503 with the reason.

That last one matters: a container that crash-loops before binding a port cannot be diagnosed. This one serves `/health`, `/health/ready` and `/metrics` on a broken deploy.

Liveness and readiness are separate. `/health/live` reports only that the process is up; `/health/ready` returns 503 until the index is loaded. Conflating them causes a classic outage — the orchestrator kills containers that are merely still warming, and never converges.

Index consistency is verified, not assumed. The vector matrix and the column store are both indexed by row position, so a mismatch would return the wrong companies with full confidence. A `RowMap` translates between them through ids. This also fixed a real bug: ingesting one company used to disable semantic search for all 50,000, because the row counts no longer matched and the matrix was discarded wholesale. Now new arrivals simply sit out semantic ranking until the next build.

Bounded resources. Search has a hard timeout (504 rather than a held worker). The rate limiter's client table is capped so the limiter cannot itself become a memory-exhaustion vector. Metric labels use templated route paths, so cardinality stays bounded no matter how many ids are hit.

One worker per container, because each process holds its own 77MB matrix, column arrays and ONNX session — workers multiply memory rather than sharing it, and two on a 512MB instance is an OOM. Scale with replicas.

---

10. Security

- **API keys**, compared with `hmac.compare_digest` so response timing does not leak how much of a guessed key was right. Reads are public by default so the deployed demo is explorable; writes always require a key once one is configured. `REQUIRE_AUTH_FOR_READS` locks reads down too.
- **Rate limiting**: in-process token bucket, per key or forwarded IP. Honest limitation — it is per-process, so N replicas means N× the limit, and it resets on restart. It exists to stop one client saturating a shared vCPU, not to enforce a quota. Multi-replica deployments should move it to Redis.
- **Injection**: FTS5 syntax is stripped and quoted; all SQL is parameterised; the UI renders exclusively via `textContent`, since company names and descriptions are attacker-controlled through the ingestion endpoint.
- **Error responses never leak internals.** One envelope, a `request_id` for correlation, tracebacks to logs only.
- Container runs as an unprivileged user.

---

11. What I would do next

Ordered by value, honestly assessed:

1. **Relevance evaluation.** There is no labelled relevance set, so ranking quality is argued from spot checks and the constraint-satisfaction check in the benchmark, not measured. A few hundred judged query-document pairs would turn weight tuning from taste into measurement. This is the largest gap.
2. **LLM query parsing behind a cache**, for the phrasings the rules miss — keeping the rules as the synchronous fallback.
3. **Learned reranking** over the top ~50 fused candidates. Little to learn on this corpus; considerable on a real one.
4. **Incremental index maintenance**, so ingestion does not rebuild the column store.
5. **Distributed rate limiting and shared caching** (Redis) once there is more than one replica.

Known limitations

- Revenue is bucketed in the source, so "revenue over 200M" matches the whole 100M–500M bucket. Interval **overlap** is used rather than containment — a recall choice, since requiring the whole bucket to satisfy the predicate would silently drop correct matches. The imprecision is inherent to the data and is surfaced in `parsed.revenue_buckets`.
- Pagination beyond the candidate pool (400 by default) degrades: the pool expands to cover `offset + limit`, but very deep paging re-ranks a large candidate set on every request.
- The relaxation order is a fixed heuristic, not learned. It is right for these queries; a different domain might rank centrality differently.
- "Startup" is recognised but deliberately **not** enforced as a size filter — 90% of this corpus has 500+ employees, so treating it literally would empty most result sets over a word people use loosely.